



TVWS-DB Configuration Manual.

Version 1.0.0

Edilson Filho, Thiago Nóbrega e Carlos Silva

Correspondent: edilsonfilho@lesc.ufc.br

Contents

1. Introduction	3
2. API Modules	3
2.1. All parts	3
2.2. Sub-modules	4
2.3. API Services	5
3. PAWS Messages	6
3.1. Function of the id Field	6
3.2. Importance of the id Field	6
3.3. Initialization	6
3.3.1. INIT_REQ	7
3.3.2. INIT_RESP	8
3.3.3. Testing	9
3.4. Available Spectrum Query	10
3.4.1. AVAIL_SPECTRUM_REQ	10
3.4.2. AVAIL_SPECTRUM_RESP	16
3.4.3. SPECTRUM_USE_NOTIFY	26
4. Customizations	28
4.1. Antenna	28
4.1.1. AntennaCharacteristics	28
4.1.2. antennaPolarization	30
4.1.3. antennaRadiationPattern	31
4.1.4. antennaGain	32
4.2. Sessions	32
4.3. Authentication	33
4.3.1. Terminology and Core Concepts	34
4.3.2. Token Assignment for Operators and Devices	34
4.3.3. Token Database Schema	34
4.3.4. Operator Checkin Flow	35
4.3.5. Device Token Assignment	36
4.3.6. Authentication and PAWS messages	37
4.3.7. Practical Example	38
4.3.8. AVAIL_SPECTRUM_REQ	38
5. Administrative Services	38
6. Deploy	44
6.1. Configuration file	44
6.1.1. blockchain	44
6.1.2. server	45
6.1.3. authentication	46

6.1.4. database	46
6.1.5. test	47
6.1.6. propagationModule	47
6.1.7. antennaParameters	48
6.1.8. standardResponseMessage	48
6.1.9. session	48
6.1.10. csv	49
6.2. Testing	49
Referencias	55

1 Introduction

The present report presents the progress made in the development of the API that functions as a communication bridge through which messages travel between the TVWS devices and the Database Chen et al., 2015. It is organized as follows:

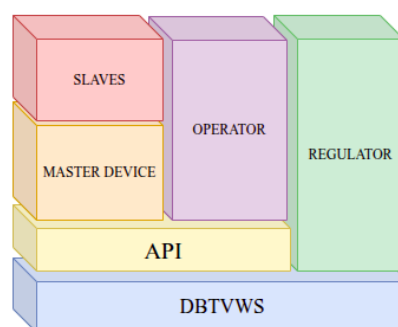
- [API Modules](#) , Apresenta an overview of the services provided by the API, as well as its operation and configuration modules.
- [Messages from PAWS](#) which describes how PAWS and RFC 7545 Chen et al., 2015 characterize the main messages exchanged between the TVWS device and the Database.
- [Customizations](#) section discusses specific adjustments implemented to tailor the solution.
- [Deploy](#) is a practical guide for instantiating the API and integrating in with the database service.

2 API Modules

This section presents the API in a modular manner, as well as its relationship with other parts of the system. Each subsection sometimes provides an overview and, at other times, a detailed description of its contents. References to the code repositories are provided to allow the reader to consult the source code when necessary.

2.1 All parts

The Figure 1 illustrates which part communicate with each of the other module of the solution.

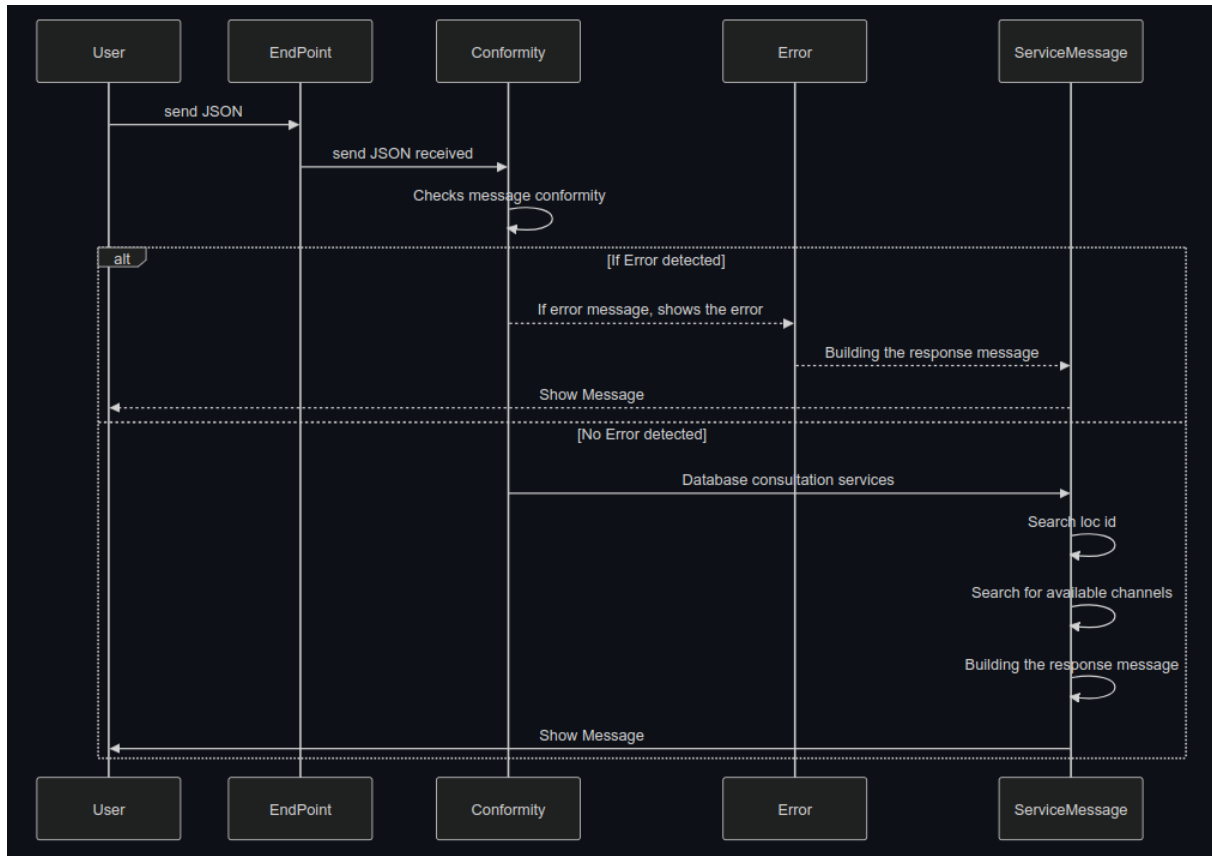


Figures 1: All partes of the TVWS Solution

The figure highlights the interfaces with which the API module interacts. With the *Regulator*, as it both controls the database and customizes and provides the API service. The *Operator*, which is regulated by the *Regulator*, manages the *Masters* and provides services to the *Slaves* when accessing the API. The *Master Device* provides services to the *Slave* and is managed by the *Operator* in order to access the API with authorization.

2.2 Sub-modules

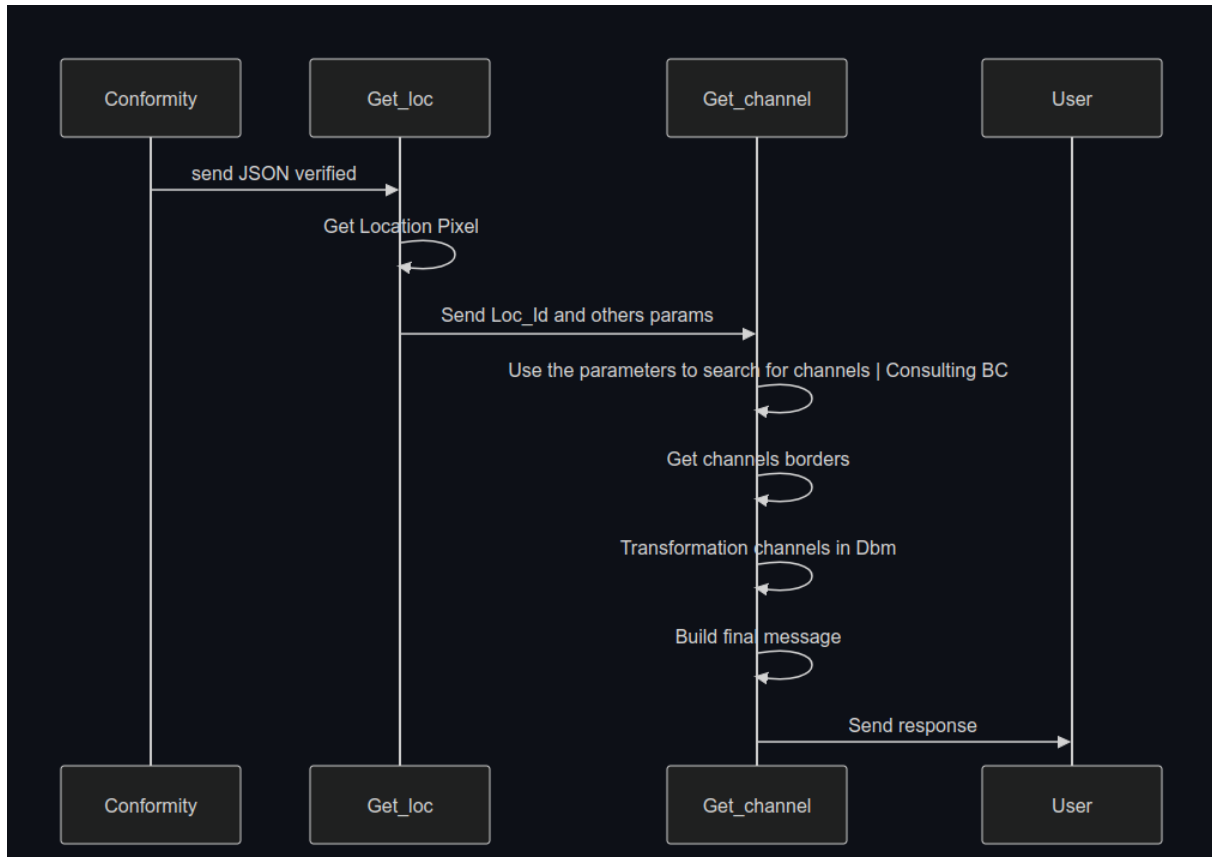
The communication between the master and the database goes through an important layer in our implementation, compliant with RFC 7545. This layer, referred to here simply as the API, handles the process of verifying messages and their *compliance* with PAWS, checks if the *location* is within the defined coverage zone, queries the *channels*, and performs the necessary calculations for channel searching. A high-level representation is presented Figure 2.



Figures 2: API Modules

User can be interpreted as a *device*. The *EndPoint* is essentially the URL used to provide the service. The *Conformity*, *Error*, and *ServiceMessage* modules are components of the API. Note that after verifying the message's conformity with RFC 7545, two scenarios are possible. The first is the error scenario, where the API will identify the error and send the device information about what went wrong, including potential solutions indicated in the message itself (such as missing or incorrect parameters). The second scenario is a successful conformity verification, where the *ServiceMessage* module will carry out the following steps: i) check the pixel on the map and return the requester's *loc_id* (which will be used to verify service coverage—if no *loc_id* is found for the requester, an error message will notify that no coverage is available for that location); and ii), after identification or *loc_id*, is the *search for available channels* and the necessary calculations for coverage, interference protection, and other results. The final step of this module is *Building the response message* (in this case, *AVAIL_SPEC_RESP*) to send to *User*.

The Figure 3 presents, in more detail, the interaction of the modules and the calculations steps. The *ServiceMessage* is divided into a) *Get_loc* and b) *Get_channel*. As mentioned earlier, *Get_loc* searches the map for the requester's unique location identifier. If successful, this value is sent to the service that will perform the *Blockchain* (BC) query and calculations such as *Get_channels borders*, *Transformation channels in dBm*, and *Build final message*.



Figures 3: ServiceMessage sub-modules

The API is multi-platform: In addition to providing the services required to respond to PAWS messages, a range of other functionalities is offered through a set of routes in an environment referred to as [administrative](#).

2.3 API Services

The API was developed to provide support and services for both PAWS messages and solution management and integration services, referred to as administrative services. Accordingly, in addition to the main URL, the path segments */paws* and */admin* direct the API to handle PAWS messages and management requests, respectively.

3 PAWS Messages

To obtain the available spectrum from the geolocation database, a master device sends a request within the *available spectrum query procedure* that contains its location and any parameters required by the ruleset (e.g., device identifier, capabilities, and characteristics). The geolocation database returns a response that describes which frequencies are available, at what permissible operating power levels, and a schedule of when they are available. In the PAWS protocol (defined in RFC 7545), messages are encoded in JSON, and the *id* field is a mandatory field for all requests and responses following version 2.0 of the protocol.

Note

The API focuses on responding to the requester. That is, in the case of a message where the locations of the slave and master are sent, the API will consider the location of the antenna to respond, including the antenna information.

3.1 Function of the id Field

The *id* field in PAWS messages serves as a *unique identifier for the request or response*¹. Its primary function is to allow the client (White-Space device) and the server (White-Space database) to correctly associate a response with a previous request. Essentially, it ensures that both parties can unequivocally track interactions between the client and the database, especially in scenarios where multiple messages are being exchanged simultaneously. Structure and Usage:

- **Request:** When the device sends a request to the database, it includes an "id" field containing a unique value for that specific request.
- **Response:** The database, when responding to the request, includes the same "id" value in its response, allowing the client device to know which request the response refers to.

3.2 Importance of the id Field

The *id* field is essential in distributed systems and *JSON-RPC*² message exchanges, like in PAWS, to ensure that communication is accurate and that the client can easily track its requests and corresponding responses. This is particularly important in scenarios with multiple requests, allowing the device to maintain control over each interaction with the spectrum database.

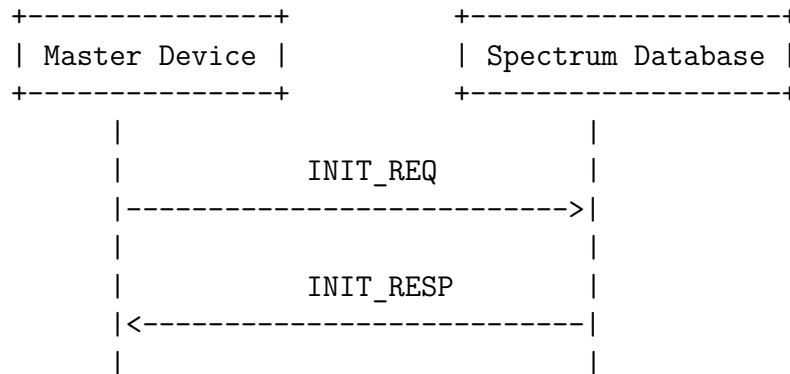
3.3 Initialization

The RFC recommends that the initialization procedure informs the Master Device of specific ruleset supported by the database, such as threshold distances, antenna parameters, time

¹See Sec 6.1 in <https://www.rfc-editor.org/rfc/rfc7545.html#page-13>

²JSON-RPC is a lightweight and simple remote procedure call (RPC) protocol based on the JSON.

periods beyond which the device must update its available-spectrum data. Figure 4 present the flow of messages.



Figures 4: Sequence of messages exchanged between Master Device and Spectrum Database.

This message has two parts:

- **INIT_REQ** : initialization request message sent by the Master Device
- **INIT_RESP** : initialization response message sent by the Spectrum Database

In the following subsections, we present our implementation of the INIT Request and Response messages as defined in RFC 7545³. Subsection 4.2 delves into the mechanisms established to maintain a history of all messages exchanged between peer devices, stateless messages named sections. These sections are delineated by specific events or message types, such as the initiation, that will be re-used in the subsequent messages.

3.3.1. INIT_REQ

The *INIT_REQ* is an initialization request message sent by a White Space Device (WSD) to the Spectrum Database Service (SBS), as defined in RFC 7545. The purpose of this request is to allow the device to obtain information about the availability of TV White Space (TVWS) spectrum in its geographic area and initiate the process of operating in that spectrum. The *INIT_REQ* is the first message exchanged in the protocol to establish a connection between the WSD and the database. The *INIT_REQ* follows the following format:

```

1 {
2   "jsonrpc": "2.0",
3   "method": "spectrum.paws.init",
4   "params": {
5     "type": "INIT_REQ",
6     "version": "1.0",
7     "deviceDesc": {
8       "serialNumber": "4fbf-bae3-c8c5706745e3",
9       "fccId": "94d8610a-9eea",
10      "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",

```

³<https://www.rfc-editor.org/rfc/rfc7545.html>

```

11     "modelId": "xpto-123-123-12",
12     "rulesetIds": ["FccTvBandWhiteSpace-2010"]
13 },
14     "location": {
15         "point": {
16             "center": {
17                 "latitude": -3.752542,
18                 "longitude": -38.556012
19             }
20         }
21     },
22 },
23     "id": "2026.1a"
24 }

```

3.3.2. INIT_RESP

The *INIT_RESP* is a message sent by the Spectrum Database Service (SBS) in response to an initialization request *INIT_REQ* from a White Space Device (WSD). The *INIT_RESP* message confirms whether the initialization request was successfully processed and provides information about the available spectrum for the device.

The *INIT_RESP* follows the following format:

```

1 {
2     "jsonrpc": "2.0",
3     "result": {
4         "type": "INIT_RESP",
5         "version": "1.0",
6         "rulesetInfos": [
7             {
8                 "authority": "br",
9                 "rulesetId": "FccTvBandWhiteSpace-2010",
10                "maxLocationChange": 100,
11                "maxPollingSecs": 86400
12            }
13        ]
14    },
15    "id": "10",
16    "version": "2026.6a"
17 }

```

The *rulesetInfos* field, according to RFC 7545, contains parameters for the ruleset of a regulatory domain that is communicated using the Initialization (5.6)⁴. In addition to the *authority* and *rulesetIds*, it includes optional response fields: *maxLocationChange*⁵ and *maxPollingSecs*⁶.

⁴<https://www.rfc-editor.org/rfc/rfc7545.html#section-5.6>

⁵The maximum location change in meters is REQUIRED for the Initialization Response

⁶The maximum duration, in seconds, between requests for available spectrum is REQUIRED for the Initialization Response

3.3.3. Testing

To use the API, two main formats can be used: i) Use the API URL and send the parameters directly from your system, as part of your integrated code, or ii) use calls via *curl*.

The geolocation database is exposed to the outside world via a Web server. For this testing period, the Web server was hosted in a public address at <http://200.160.6.8/api/> and it was implemented using the Nginx⁷ tools. For testing with *curl* we present two examples:

Option 1 From file, where PATH/TO_FILE.json is the path to a JavaScript Object Notation (JSON) file

```
curl -X POST URL_HERE \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer <token>" \  
--data @PATH/TO_FILE.json
```

Option 2 Inline, here in INIT_REQ is just an example

```
curl -X POST URL_HERE \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer <token>" \  
-d '{"jsonrpc":"2.0","method":"spectrum.paws.init","params":{"type":"  
INIT_REQ","version":"1.0","deviceDesc":{"serialNumber":"4fbf-bae3-  
c8c5706745e3","fccId":"94d8610a-9eea","manufacturerId":"123-ABCD-4  
fbf-bae3-c8c5706745e3","modelId":"xpto-123-123-12","rulesetIds":["  
FccTvBandWhiteSpace-2010"]},"location":{"point":{"center":{"latitude  
":-3.752542,"longitude":-38.556012}}},"id":"2026.1a"}'
```

Considerations Some considerations are important for handling this message:

- If the rulesetIds is not sent as an array, the API will return the rule it follows: *FccTvBandWhiteSpace-2010*
- The location used in this message is that of the master
- the *id* field present in messages like *INIT_REQ* or *AVAIL_SPECTRUM_REQ* is assigned by the system that is sending the message, i.e., the White Space Device (WSD), whether it is a master or a slave. This field is used to link a request to its corresponding response.

Params Contains the parameters for the request. For the *INIT_REQ*, it includes the following subfields:

- *serialNumber*: The serial number of the device, used to uniquely identify it.

⁷<https://nginx.org/en/>.

- *fccId*: The identifier of the device registered with the FCC⁸.
- *rulesetIds*: Defines the applicable rulesets for the device, here *FccTvBandWhiteSpace-2010*, indicating it is operating under the FCC 2010 rules for White Space.
- *id*: A unique identifier for the request, used to correlate the response.

3.4 Available Spectrum Query

The *AVAIL_SPECTRUM_REQ* message is a request sent by a White Space Device (WSD) to a Spectrum Database Service (SBS) to query the availability of TV White Space (TVWS) spectrum in the device's current location. This section outlines our implementation of the Available Spectrum Query messages, as specified in RFC 7545. Figure 5 present the flow of messages. Our implementation currently supports the:

- *AVAIL_SPECTRUM_REQ*,
- *AVAIL_SPECTRUM_RESP*,
- *SPECTRUM_USE_NOTIFY* (simplified) messages.

Table 1 show the messages implemented in our solution.

Tables 1: Available Spectrum Query Implementation Status

Message	Implementation
<i>AVAIL_SPECTRUM_REQ</i>	Ok
<i>AVAIL_SPECTRUM_RESP</i>	Ok
<i>AVAIL_SPECTRUM_BATCH_REQ</i>	
<i>AVAIL_SPECTRUM_BATCH_RESP</i>	

3.4.1. AVAIL_SPECTRUM_REQ

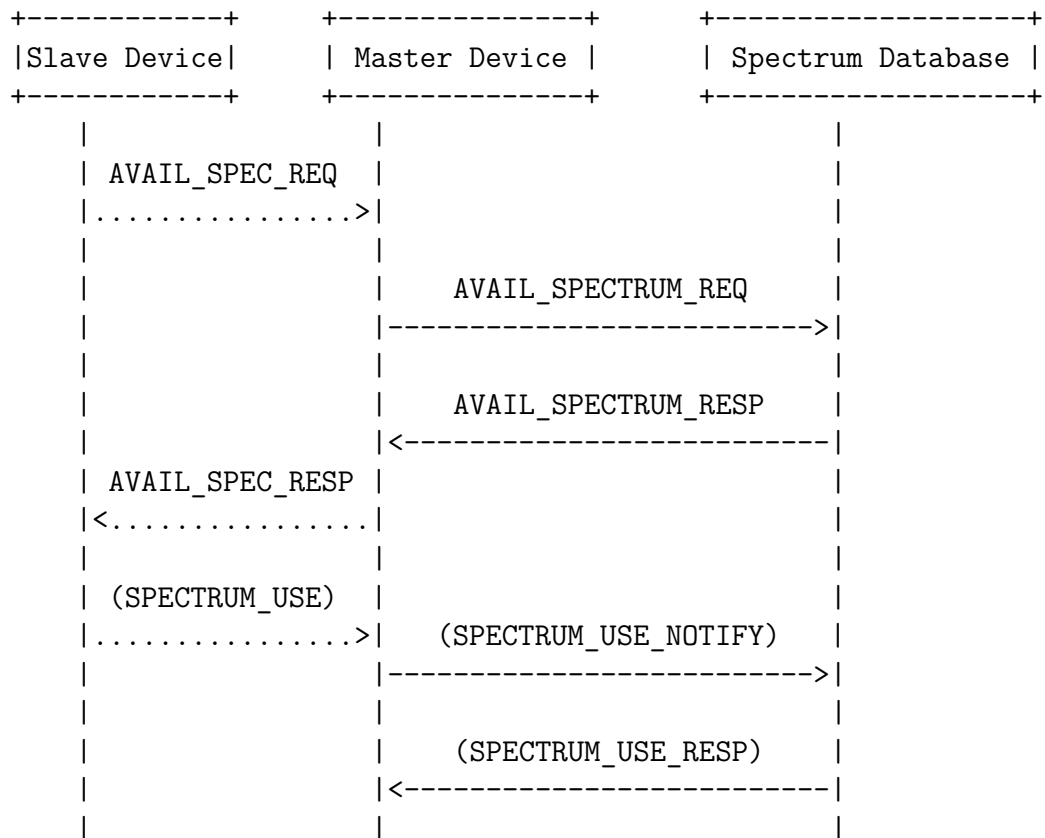
The *AVAIL_SPECTRUM_REQ* follows the following format:

```

1 {
2   "jsonrpc": "2.0",
3   "method": "spectrum.paws.getSpectrum",
4   "params": {
5     "type": "AVAIL_SPECTRUM_REQ",
6     "version": "1.0",
7     "deviceDesc": {
8       "serialNumber": "4fbf-bae3-c8c5706745e3",
9       "fccId": "94d8610a-9eea",
10      "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",

```

⁸Federal Communications Commission



Figures 5: Sequence of messages exchanged in Avail Spectrum Query

```

11     "modelId": "xpto-123-123-123",
12     "rulesetIds": [
13         "FccTvBandWhiteSpace-2010"
14     ],
15 },
16     "masterDeviceDesc": {
17         "serialNumber": "4fbf-bae3-c8c5706745e3",
18         "fccId": "94d8610a-9eea",
19         "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",
20         "modelId": "xpto-123-123-123",
21         "rulesetIds": [
22             "FccTvBandWhiteSpace-2010"
23         ]
24     },
25     "location": {
26         "point": {
27             "center": {
28                 "latitude": -4.97139, "longitude": -39.01528
29             }
30         }
31     },

```

```
32  "deviceOwner":{
33    "owner":{
34      "vcard":[
35        [
36          "version",
37          {
38
39          },
40          "text",
41          "4.0"
42        ],
43        [
44          "fn",
45          {
46
47          },
48          "text",
49          "Fractal Networks"
50        ],
51        [
52          "org",
53          {
54
55          },
56          "text",
57          "Fractal Networks"
58        ],
59        [
60          "adr",
61          {
62
63          },
64          "text",
65          [
66            "",
67            "",
68            "123 Innovation Ave",
69            "San Francisco",
70            "CA",
71            "94107",
72            "USA"
73          ]
74        ],
75        [
76          "email",
77          {
78
79          },
```

```
80         "text",
81         "contact@fractalfn.com"
82     ],
83     [
84         "tel",
85         {
86
87         },
88         "uri",
89         "tel:+1-415-555-0100"
90     ]
91 ]
92 },
93 "operator":{
94     "vcard":[
95         [
96             "version",
97             {
98
99             },
100            "text",
101            "4.0"
102        ],
103        [
104            "fn",
105            {
106
107            },
108            "text",
109            "Jane Doe"
110        ],
111        [
112            "org",
113            {
114
115            },
116            "text",
117            "Fractal Networks"
118        ],
119        [
120            "adr",
121            {
122
123            },
124            "text",
125            [
126                "",
127                "",
```

Pág. 14 - 55

```

171     }
172   }
173 },
174   "id": "10"
175 }

```

The types of variables are:

Field	Data Type	Unit/Format
"jsonrpc"	String	Fixed value: "2.0"
"method"	String	"spectrum.paws.getSpectrum"
"params"	Object	N/A
"type"	String	AVAIL_SPECTRUM_REQ
"version"	String	"1.0"
"deviceDesc"	Object	N/A
"serialNumber"	String	Alphanumeric ID
"fccId"	String	Alphanumeric (e.g., FCC ID)
"rulesetIds"	Array of Strings	N/A
"location"	Object	N/A
"point"	Object	N/A
"center"	Object	N/A
"latitude"	Number (Float)	Degrees (−90 to 90)
"longitude"	Number (Float)	Degrees (−180 to 180)
"antenna"	Object	N/A
"height"	Number (Integer)	Meters
"heightType"	String	AGL or ASL
"antennaDirection"	Object	N/A
"zenith"	Number (Float)	Degrees (0 to 180)
"azimuth"	Number (Float)	Degrees (0 to 360)
"antennaRadiationPattern"	Array of Numbers (Float)	dBi (≤ 0 dBi), 72 values per polarisation
"antennaGain"	Number (Float)	dBi (0 to 50), optional
"antennaPolarization"	String	"v"(vertical) or "h"(horizontal)
"masterDeviceLocation"	Object	N/A
"point"	Object	N/A
"center"	Object	N/A
"latitude"	Number (Float)	Degrees (−90 to 90)
"longitude"	Number (Float)	Degrees (−180 to 180)
"id"	String	Unique Identifier

Tables 2: Explanation of the Fields of message

Antenna Gain Validation (antennaGain). The antenna object in AVAIL_SPECTRUM_REQ accepts an optional antennaGain field that allows the device to report the peak gain of its antenna. When this field is present, the compliance module (server/messages/compliance.js) enforces the following rules before forwarding the request to the spectrum service:

- The value must be a finite numeric type. Strings, booleans, and null are rejected.
- The value must be within the range $[0, 50]$ dBi, configurable via `antennaParameters.minGainDbi` and `antennaParameters.maxGainDbi` in `config/default.json`.
- If either condition is violated, the server returns error `-202 (INVALID_VALUE)` and the field path `params.antenna.antennaGain` in the error detail. The request is not processed further.
- If the field is absent, validation is skipped entirely and the request proceeds normally.

The accepted range (0–50 dBi) covers the full span of practical TVWS antenna designs. Values outside this range are considered physically unrealistic and are rejected to prevent misconfigured devices from receiving incorrect spectrum allocations.

Antenna Radiation Pattern Validation (`antennaRadiationPattern`). The `antennaRadiationPattern` array carries the relative directional gain of the antenna, normalised to its peak. The compliance module enforces the following rules on every value in every sub-array:

- Each element must be a finite numeric type. Non-numeric or non-finite values return error `-202`.
- Each element must be ≤ 0 dBi. Because the pattern is expressed relative to the antenna's peak gain, 0 dBi represents the direction of maximum radiation; all other directions must have a negative (attenuated) value. Any value greater than 0 dBi is physically inconsistent and returns error `-202 (INVALID_VALUE)` with the field path `params.antenna.antennaRadiationPattern`.

Positional convention. Index 0 of each sub-array corresponds to 0° , which is aligned with the antenna zenith direction. The zenith uses **true geographic north** as its azimuth reference. Subsequent indices advance in **clockwise** order in 5° steps: index 1 = 5° , index 2 = 10° , ..., index 71 = 355° .

3.4.2. AVAIL_SPECTRUM_RESP

```
1 {
2   "jsonrpc": "2.0",
3   "result": {
4     "type": "AVAIL_SPECTRUM_RESP",
5     "version": "1.0",
6     "timestamp": "2026-06-03T08:20:12Z",
7     "deviceDesc": {
8       "serialNumber": "4fbf-bae3-c8c5706745e3",
9       "fccId": "94d8610a-9eea",
10      "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",
11      "modelId": "xpto-123-123-123",
12      "rulesetIds": [
13        "FccTvBandWhiteSpace-2010"
```

```
14     ]
15 },
16 "spectrumSpecs":[
17     {
18         "rulesetInfo":{
19             "authority":"ANATEL-BR",
20             "rulesetId":"FccTvBandWhiteSpace-2010",
21             "maxLocationChange":100,
22             "maxPollingSecs":86400
23         },
24         "needsSpectrumReport":false,
25         "maxTotalBwHz":6000000,
26         "spectrumSchedules":[
27             {
28                 "eventTime":{
29                     "startTime":"2026-06-03T08:20:12Z",
30                     "stopTime":"2026-06-03T08:20:12Z"
31                 },
32                 "spectra":[
33                     {
34                         "resolutionBwHz":6000000,
35                         "profiles":[
36                             [
37                                 [
38                                     {
39                                         "hz":54000000,
40                                         "dbm":14.38
41                                     },
42                                     {
43                                         "hz":60000000,
44                                         "dbm":14.38
45                                     },
46                                     {
47                                         "hz":60000000,
48                                         "dbm":14.86
49                                     },
50                                     {
51                                         "hz":66000000,
52                                         "dbm":14.86
53                                     },
54                                     {
55                                         "hz":66000000,
56                                         "dbm":15.25
57                                     },
58                                     {
59                                         "hz":72000000,
60                                         "dbm":15.25
61                                     }

```

```
62     ],
63     [
64         {
65             "hz":174000000,
66             "dbm":19.84
67         },
68         {
69             "hz":180000000,
70             "dbm":19.84
71         },
72         {
73             "hz":180000000,
74             "dbm":20.04
75         },
76         {
77             "hz":186000000,
78             "dbm":20.04
79         },
80         {
81             "hz":186000000,
82             "dbm":20.23
83         },
84         {
85             "hz":192000000,
86             "dbm":20.23
87         },
88         {
89             "hz":192000000,
90             "dbm":20.43
91         },
92         {
93             "hz":198000000,
94             "dbm":20.43
95         },
96         {
97             "hz":198000000,
98             "dbm":20.53
99         },
100        {
101            "hz":204000000,
102            "dbm":20.53
103        },
104        {
105            "hz":204000000,
106            "dbm":20.72
107        },
108        {
109            "hz":210000000,
```

```
110         "dbm":20.72
111     },
112     {
113         "hz":210000000,
114         "dbm":20.92
115     },
116     {
117         "hz":216000000,
118         "dbm":20.92
119     }
120 ],
121 [
122     {
123         "hz":470000000,
124         "dbm":24.73
125     },
126     {
127         "hz":476000000,
128         "dbm":24.73
129     },
130     {
131         "hz":476000000,
132         "dbm":24.82
133     },
134     {
135         "hz":482000000,
136         "dbm":24.82
137     },
138     {
139         "hz":482000000,
140         "dbm":24.82
141     },
142     {
143         "hz":488000000,
144         "dbm":24.82
145     },
146     {
147         "hz":488000000,
148         "dbm":24.92
149     },
150     {
151         "hz":494000000,
152         "dbm":24.92
153     },
154     {
155         "hz":494000000,
156         "dbm":24.92
157     },
158 ],
```

```
158      {
159          "hz":500000000,
160          "dbm":24.92
161      },
162      {
163          "hz":500000000,
164          "dbm":25.02
165      },
166      {
167          "hz":506000000,
168          "dbm":25.02
169      },
170      {
171          "hz":506000000,
172          "dbm":25.02
173      },
174      {
175          "hz":512000000,
176          "dbm":25.02
177      },
178      {
179          "hz":512000000,
180          "dbm":25.12
181      },
182      {
183          "hz":518000000,
184          "dbm":25.12
185      },
186      {
187          "hz":518000000,
188          "dbm":25.12
189      },
190      {
191          "hz":524000000,
192          "dbm":25.12
193      },
194      {
195          "hz":524000000,
196          "dbm":25.21
197      },
198      {
199          "hz":530000000,
200          "dbm":25.21
201      },
202      {
203          "hz":530000000,
204          "dbm":25.21
205      },
```

```
206      {
207          "hz":536000000,
208          "dbm":25.21
209      },
210      {
211          "hz":536000000,
212          "dbm":25.31
213      },
214      {
215          "hz":542000000,
216          "dbm":25.31
217      },
218      {
219          "hz":542000000,
220          "dbm":25.31
221      },
222      {
223          "hz":548000000,
224          "dbm":25.31
225      },
226      {
227          "hz":548000000,
228          "dbm":25.41
229      },
230      {
231          "hz":554000000,
232          "dbm":25.41
233      },
234      {
235          "hz":554000000,
236          "dbm":25.41
237      },
238      {
239          "hz":560000000,
240          "dbm":25.41
241      },
242      {
243          "hz":560000000,
244          "dbm":25.51
245      },
246      {
247          "hz":566000000,
248          "dbm":25.51
249      },
250      {
251          "hz":566000000,
252          "dbm":25.51
253      },
```

```
254      {
255          "hz": 572000000,
256          "dbm": 25.51
257      },
258      {
259          "hz": 572000000,
260          "dbm": 25.51
261      },
262      {
263          "hz": 578000000,
264          "dbm": 25.51
265      },
266      {
267          "hz": 578000000,
268          "dbm": 25.61
269      },
270      {
271          "hz": 584000000,
272          "dbm": 25.61
273      },
274      {
275          "hz": 584000000,
276          "dbm": 25.61
277      },
278      {
279          "hz": 590000000,
280          "dbm": 25.61
281      },
282      {
283          "hz": 590000000,
284          "dbm": 25.7
285      },
286      {
287          "hz": 596000000,
288          "dbm": 25.7
289      },
290      {
291          "hz": 596000000,
292          "dbm": 25.7
293      },
294      {
295          "hz": 602000000,
296          "dbm": 25.7
297      },
298      {
299          "hz": 602000000,
300          "dbm": 25.7
301      },
```

```
302      {
303        "hz": 608000000,
304        "dbm": 25.7
305      },
306      {
307        "hz": 614000000,
308        "dbm": 25.8
309      },
310      {
311        "hz": 620000000,
312        "dbm": 25.8
313      },
314      {
315        "hz": 620000000,
316        "dbm": 25.8
317      },
318      {
319        "hz": 626000000,
320        "dbm": 25.8
321      },
322      {
323        "hz": 626000000,
324        "dbm": 25.9
325      },
326      {
327        "hz": 632000000,
328        "dbm": 25.9
329      },
330      {
331        "hz": 632000000,
332        "dbm": 25.9
333      },
334      {
335        "hz": 638000000,
336        "dbm": 25.9
337      },
338      {
339        "hz": 638000000,
340        "dbm": 25.9
341      },
342      {
343        "hz": 644000000,
344        "dbm": 25.9
345      },
346      {
347        "hz": 644000000,
348        "dbm": 26
349      },
```

```
350      {
351          "hz": 650000000,
352          "dbm": 26
353      },
354      {
355          "hz": 650000000,
356          "dbm": 26
357      },
358      {
359          "hz": 656000000,
360          "dbm": 26
361      },
362      {
363          "hz": 656000000,
364          "dbm": 26
365      },
366      {
367          "hz": 662000000,
368          "dbm": 26
369      },
370      {
371          "hz": 662000000,
372          "dbm": 26.09
373      },
374      {
375          "hz": 668000000,
376          "dbm": 26.09
377      },
378      {
379          "hz": 668000000,
380          "dbm": 26.09
381      },
382      {
383          "hz": 674000000,
384          "dbm": 26.09
385      },
386      {
387          "hz": 674000000,
388          "dbm": 26.09
389      },
390      {
391          "hz": 680000000,
392          "dbm": 26.09
393      },
394      {
395          "hz": 680000000,
396          "dbm": 26.19
397      },
```

```

398      {
399          "hz":686000000,
400          "dbm":26.19
401      },
402      {
403          "hz":686000000,
404          "dbm":26.19
405      },
406      {
407          "hz":692000000,
408          "dbm":26.19
409      },
410      {
411          "hz":692000000,
412          "dbm":26.19
413      },
414      {
415          "hz":698000000,
416          "dbm":26.19
417      }
418    ],
419  ],
420  ],
421  },
422  ],
423  },
424  ],
425  },
426  ],
427  },
428  "id":"10",
429  "version":"2026.6a"
430  }

```

Testing Option 1 (from file, where PATH/TO_FILE.json is the path to a JSON file)

```

curl -X POST URL_HERE \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  --data @PATH/TO_FILE.json

```

Option 2 (inline, here in AVAIL_SPECTRUM_REQ is just an example)

```

curl -X POST URL_HERE \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \

```

```
-d '{"jsonrpc":"2.0","method":"spectrum.paws.getSpectrum","params":{"type":"AVAIL_SPECTRUM_REQ","version":"1.0","deviceDesc":{"serialNumber":"4fbf-bae3-c8c5706745e3","fccId":"94d8610a-9eea","manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3","modelId":"xpto-123-123-123","rulesetIds":["FccTvBandWhiteSpace-2010"]},"location":{"point":{"center":{"latitude":-4.978806,"longitude":-39.056607}}},"deviceOwner":{"owner":{"vcard":[{"version",{},"text","4.0"},{"fn",{},"text","Fractal Networks"},{"org",{},"text","Fractal Networks"},{"email",{},"text","contact@fractalfn.com"},{"tel",{},"uri","tel:+1-415-555-0100"}]},"operator":{"vcard":[{"version",{},"text","4.0"},{"fn",{},"text","Jane Doe"},{"org",{},"text","Fractal Networks"},{"email",{},"text","jane.doe@fractalfn.com"},{"tel",{},"uri","tel:+1-415-555-0101"}]},"antenna":{"height":70,"heightType":"AGL"}},"id":"2026.1a"}'
```

3.4.3. SPECTRUM_USE_NOTIFY

SPECTRUM_USE_NOTIFY message is used to notify the Spectrum Database Service (SBS) about the start or end of a White Space Device's (WSD) operation on a TV White Space (TVWS) spectrum channel.

Purpose of *SPECTRUM_USE_NOTIFY*

- The *SPECTRUM_USE_NOTIFY* message is sent by a WSD to inform the SBS that it has begun or stopped using the spectrum at a specific location and frequency range.
- The primary purpose is to keep the spectrum database updated about device operations, enabling the SBS to monitor spectrum usage across various geographical areas and frequency bands.

Usage Context After receiving a positive response to a spectrum request, such as an *AVAIL_SPECTRUM_RESP*, the WSD may start using the allocated TVWS spectrum. Upon starting this operation, the device sends a *SPECTRUM_USE_NOTIFY* to the SBS, indicating it has begun using the spectrum

Key fields and their descriptions

- *params*: This section contains all the parameters required for the notification, in this case: *SPECTRUM_USE_NOTIFY*
- *location*: Provides the geographical location of the notifying device.
- *spectra*: Details the frequency spectrum being used by the device: i) *resolutionBwHz* is the resolution bandwidth in Hz and ii) *profiles*, contains a list of frequency power profile (in hz and dbm). The Table 3 presents the data types and the units used in spectra.

The *SPECTRUM_USE_NOTIFY* follows the following format:

```
1 {
2   "jsonrpc": "2.0",
3   "method": "spectrum.paws.notifySpectrumUse",
4   "params": {
5     "type": "SPECTRUM_USE_NOTIFY",
6     "version": "1.0",
7     "deviceDesc": {
8       "serialNumber": "4fbf-bae3-c8c5706745e3",
9       "fccId": "94d8610a-9eea",
10      "rulesetIds": [
11        "FccTvBandWhiteSpace-2010"
12      ]
13    },
14    "location": {
15      "point": {
16        "center": {
17          "latitude": -3.7933105,
18          "longitude": -38.6020162
19        }
20      }
21    },
22    "masterDeviceDesc": {
23      "serialNumber": "4fbf-bae3-c8c5706745e3",
24      "fccId": "94d8610a-9eea",
25      "rulesetIds": [
26        "FccTvBandWhiteSpace-2010"
27      ]
28    },
29    "masterDeviceLocation": {
30      "point": {
31        "center": {
32          "latitude": -3.7933105,
33          "longitude": -38.6020162
34        }
35      }
36    },
37    "spectra": [
38      {
39        "resolutionBwHz": 6000000,
40        "profiles": [
41          [
42            {
43              "hz": 54000000,
44              "dbm": 29
45            },
46            {
47              "hz": 60000000,
```



Field	Data Type	Unit/Format
resolutionBwHz	Integer/Float	Hertz (Hz)
profiles	Array of Arrays of Objects	List of frequency-power profiles
hz (within profiles)	Integer/Float	Hertz (Hz)
dbm (within profiles)	Integer/Float	Decibel-milliwatts (dBm)

Tables 3: Explanation of the spectra Fields

4 Customizations

Some customizations were implemented in the TVWS support provided by the API, notably in the areas of [antennas](#), [session](#) management, and [authentication](#).

4.1 Antenna

This section introduces a set of new attributes designed to enhance communication capabilities. Specifically, the *AntennaCharacteristics*⁹ parameter of the PAWS Protocol to implement an optimized usage of the spectrum.

4.1.1. AntennaCharacteristics

The basic AntennaCharacteristics of RFC 7545 covers essential parameters detailed in the schema below.

```

+-----+
|AntennaCharacteristics|
+-----+-----+
|height:float          | OPTIONAL |
|heightType:enum       | OPTIONAL |
|heightUncertainty:float| OPTIONAL |
|.....|.....|
|*characteristics:     | OPTIONAL |
|  various              |          |

```

⁹<https://www.rfc-editor.org/rfc/rfc7545.html#section-5.3>

+-----+-----+

The default schema (presented above) includes the following parameters:

- Height (height): This parameter represents the physical height of the antenna above the ground or a reference point. It is typically measured in **meters**. The height significantly influences the antenna's coverage area and interaction with its surroundings.
- Height Type (heightType): An optional field that specifies the type of height measurement. Common types include:
 - Above Ground Level (AGL)
 - Above Mean Sea Level (AMSL)
- Height Uncertainty (heightUncertainty): Another optional field that quantifies the uncertainty associated with the height measurement. It accounts for inaccuracies or variations in the height value.

The literature reports that when designing and deploying wireless communication systems, accurate modeling of antenna behavior is essential for efficient spectrum utilization and user protection. While basic parameters like antenna height provide a foundation, more is needed to capture the intricacies of real-world scenarios.

To create a more precise model¹⁰, therefore promoting better user protection (primary and secondary) and an efficacy spectrum usage, we incorporate the following optional¹¹ fields to enhance the system performance:

- antenna direction
- antenna radiation pattern
- antenna gain (extracted from the radiation pattern)
- antenna polarization

The code fragment below illustrates our implementation of the optional fields.

```

1 {
2   "jsonrpc": "2.0",
3   "method": "spectrum.paws.getSpectrum",
4   "params": {
5     "type": "AVAIL_SPECTRUM_REQ",
6     "version": "1.0",
7     (...)
8     "antenna": {
9       "height": 76,
```

¹⁰In terms of propagation: coverage and interference protection

¹¹These optional fields (antenna direction, radiation pattern, gain and polarization) are presented in section 5.3 of the RFC7545

```

10     "heightType": "AGL",
11     "antennaDirection": {
12         "zenith": 10.1,
13         "azimuth": 12.1
14     },
15     "antennaRadiationPattern": [11.1, ..., 22.2],
16     "antennaPolarization": "v"
17 },
18 (...)
19 },
20 "id": "001"
21 }

```

We use the Azimuth and Zenith to represent antenna direction. The Azimuth is an angle (double/float) determined using the geographic north (0,0).

The antennaRadiationPattern is represented by 72 relative gain values (in dBi) sampled every 5° to form a complete azimuthal radiation pattern. Index 0 corresponds to 0°, which is aligned with the antenna zenith using **true geographic north** as the reference direction. The remaining indices advance in **clockwise** order.

The Antenna Gain will be expressed in **dBi**, as a relative value normalised to the antenna peak (≤ 0 dBi).

For example, pattern[0] indicates the relative gain at 0° (true north / zenith reference) and pattern[71] indicates the relative gain at 355°.

Default Values

- if the user does not send the AntennaCharacteristics the system will consider an **isotropic antenna with zero gain**.
- If the user does not send the complete **72 positions** of the radion pattern gain, thus an isotropic antenna is assumed with gain equal to the maximum value within the array.

4.1.2. antennaPolarization

We also represent the polarization, witch affect the electromagnetic waves propagation. Using the following flags (See datatype in Table 2):

- *v* : Vertical
- *h* : Horizontal
- *e*: Elliptical
- *v,h*: Vertical and Horizontal, and it will consists an array of arrays

If the value sent in the message does not correspond to one of these two values (string and lowercase), the system will return an error message, indicating that something is wrong, as shown in the following image. If no polarization parameter is entered our system will consider the **horizontal polarazition**, *h*.

4.1.3. antennaRadiationPattern

The `antennaRadiationPattern` uses the antenna polarization as an index. If the polarization is specified as "v", "h", or "e", the array in `antennaRadiationPattern` represents the gain per radial (as described in a previous section) corresponding to that polarization.

However, if the polarization is specified as "v,h", then `antennaRadiationPattern` is defined as an array of arrays:

- The **outer array** contains two elements, one for each polarization;
- Each **inner array** contains 72 values, corresponding to the gain per radial direction.

Each value in `antennaRadiationPattern` is calculated as:

$$-10 \log_{10} \left(\left(\frac{E_x}{E_{\max}} \right)^2 \right)$$

where $x = h$ or $x = v$, representing the horizontal or vertical components of the electric field, respectively.

In the case of elliptical polarization, it is assumed that the combination of the horizontal (E_h) and vertical (E_v) electric field components for each radial direction has already been computed. The corresponding value is given by:

$$-10 \log_{10} \left(\left(\frac{E_e}{E_{\max}} \right)^2 \right) = -10 \log_{10} \left(\left(\frac{E_h}{E_{\max}} \right)^2 \times \left(\frac{E_v}{E_{\max}} \right)^2 \right)$$

Positional convention. The pattern array is indexed starting at position 0, which corresponds to 0° . This direction is aligned with the **antenna zenith**, which in turn uses **true geographic north** as its reference. Subsequent positions advance in the **clockwise** direction (the same sense as a clock face viewed from above), with each step representing an additional 5° :

- Index 0 $\rightarrow 0^\circ$ (zenith / true north reference)
- Index 1 $\rightarrow 5^\circ$
- Index 2 $\rightarrow 10^\circ$
- ...
- Index 71 $\rightarrow 355^\circ$

Value constraint. Every element of the pattern must satisfy value ≤ 0 dBi. The pattern is expressed as a *relative* gain normalised to the antenna's peak gain: 0 dBi indicates the direction of maximum radiation, and all other directions must carry a negative value (i.e., attenuation relative to the peak). The compliance module (`server/messages/compliance.js`) rejects any element greater than 0 dBi with error -202 (INVALID_VALUE).

4.1.4. antennaGain

The optional `antennaGain` field inside the `antenna` object allows the device to report the peak gain of its antenna. When present, the compliance module (`server/messages/compliance.js`) validates this field before any spectrum query is processed:

- The value must be a finite number (not a string, boolean, or `null`).
- The value must fall within the range `[0, 50]` dBi, as configured by `antennaParameters.minGainDbi` and `antennaParameters.maxGainDbi` in `config/default.json`.
- If either condition is violated, the server immediately returns error `-202 (INVALID_VALUE)` with the offending field path `params.antenna.antennaGain` and the request is not forwarded to the spectrum service.
- If the field is absent, validation is skipped and the request proceeds normally.

The accepted range (0–50 dBi) covers the full span of practical TV white-space antenna designs, from isotropic reference antennas (0 dBi) to high-gain directional arrays (up to 50 dBi). Values outside this range are considered physically unrealistic and are rejected to prevent misconfigured devices from obtaining incorrect spectrum allocations.

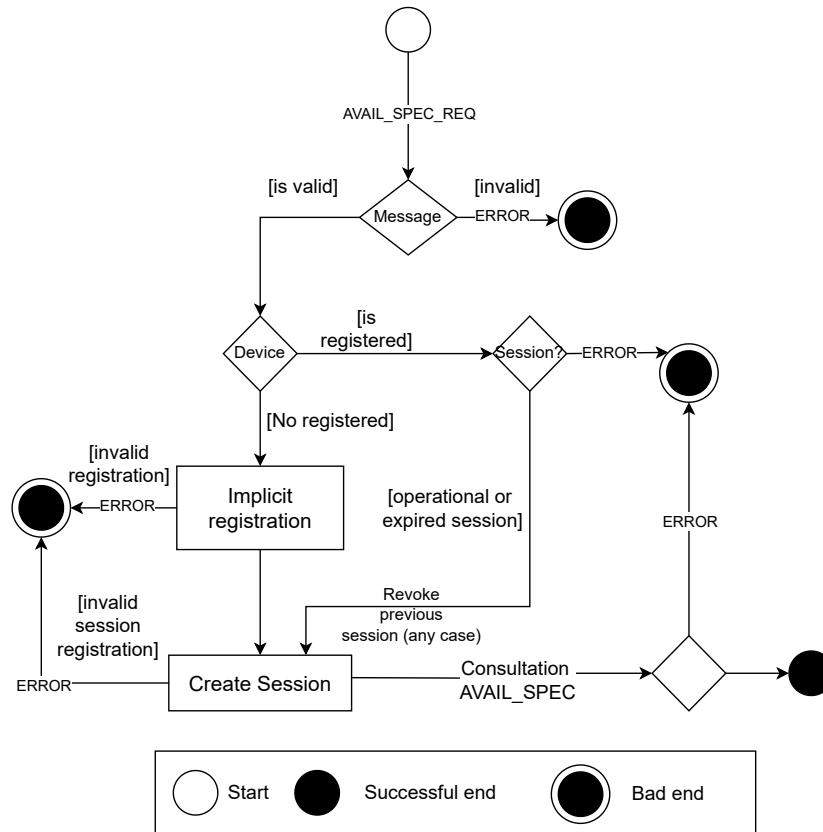
4.2 Sessions

In the context of APIs based on the PAWS (Protocol to Access White Space) standard, as defined in RFC 7545, the use of sessions plays a crucial role in efficiently and securely managing spectrum access by authorized devices.

Sessions allow for maintaining a persistent state between the spectrum database and the devices during a communication period. This is essential to ensure that decisions regarding spectrum usage are made based on the device's interaction history and context.

In our implementation, each device is identified by a unique set of attributes: i) *serialNumber*, ii) *manufacturerId* and iii) *modelId*. These identifiers are used to link a specific session to a physical device, enabling the API to accurately recognize which device is making the request, even across multiple connections over time. This provides several key benefits:

- **State Management:** Allows tracking of request and response history, helping maintain compliance with spectrum usage regulations.
- **Security and Authenticity:** Reduces the risk of fraudulent or malicious requests, since each session is tied to a validated device.
- **Response Efficiency:** Enables optimizations such as response caching or applying differentiated usage policies per session.
- **Auditing and Monitoring:** Facilitates analysis of spectrum usage patterns, detection of anomalies, and operational troubleshooting.



Figures 6: Implicit Session Creation. "Note that upon receiving the message, the API checks whether a registration already exists for that device. If no registration is found, the parameters provided in the message are used to create the device registration (implicit registration), followed by the session creation and the corresponding response to the message

See Figure 6 for an example of the implementation of implicit session creation.

The session implementation follows two possible flows: i) When a Registration_REQ is sent to the database, this is referred to as explicit session creation, or ii) When an Avail_Spectrum_Req is sent, in which case the session is created implicitly.

Since *Registration_Req* is optional, but *Avail_Spectrum_Req* is mandatory, we will present a diagram below that explains how the session is created via *Avail_Spectrum_Req* using the concept of an implicit *Registration_Req*.

4.3 Authentication

This subsection present the proposed model for authentication, registration, and *token* management within the TV White Space (TVWS) API ecosystem, inspired by 2.1.7 *Client Auth Candidate 2: Shared Secret* from the document *Brazil TVWS – Phase 2, version dated September 17, 2025*. In this context, UUID tokens will be used as the mechanism for authentication. Tokens will be assigned to both Operators and Devices, in accordance with section 2.1.7.

4.3.1. Terminology and Core Concepts

Table 4 introduces the main terms used in the TVWS architecture and their meanings. Furthermore, the table presents some blockchain-specific terms, we will not delve into these terms or the operational mechanisms underlying this technology.

Tables 4: Main terminology used in the TVWS architecture

Term	Description
Checkin	Registration of the operator to receive the auth token
wallet	Blockchain account (DBTVWS) associated with an operator.
address	Unique identifier for an account on the blockchain.

4.3.2. Token Assignment for Operators and Devices

The allocation of tokens to operators and devices involves an additional level of context, as operators are issued a credential, called a wallet, which allows them to assign tokens to devices. This section briefly presents information about the [Token Format](#), the [Operator checkin Flow](#), and how authentication will be handled during [PAWS messages](#), in Phase II.

Token Format

The authentication model uses UUID¹² v4 tokens, prefixed with metadata identifying the operator and token type.

Example:
3f4e9c80-2c47-4a9d-b8d2-f0c5bb7d5aaf
8c02c5d0-9e81-47d0-8617-7df49ce8efb3

Internally, this token is stored in a database table along with its metadata. In the [Token Database Schema](#), we present the fields of the table that stores the data for authentication.

4.3.3. Token Database Schema

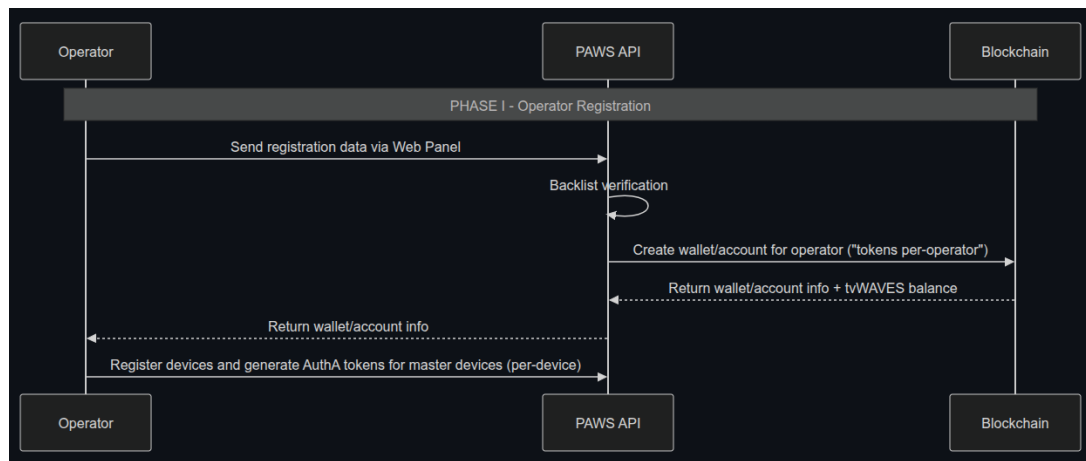
Field	Type	Description
token	string	Full token
scope	enum(operator,device)	token scope: O (operator) or D (device)
operator_id	string	vat_number without non-numeric characters
device_id	string (nullable)	Device identifier (serialNumber)
model_id	string (nullable)	modelID in deviceDesc
manufacturer_id	string	ManufacturerId in deviceDesc
created_at	date	Issue date
expires_at	date	Expiration date
revoked	boolean	Revocation flag

¹²**UUID:** random 128-bit identifier

contact	string(nullable/device)	email admin who issued the token
wallet	string (nullable)	Account in blockchain
public key	string (nullable)	Public wallet key
private key	string (nullable)	Private wallete key
balance	string	balance governance token
description	string (nullable)	Optional management description
version	string	authentication version control information

4.3.4. Operator Checkin Flow

The following diagram illustrates the operator registration and wallet creation process.



Figures 7: Operator Registration Flow

The Figure 7 illustrates the three entities involved in the operator check-in and token assignment process. Initially, the operator, via JSON and API, submits their registration data to create an account to access the database resources. The API processes this check-in message and verifies whether the CNPJ (Brazilian National Registry of Legal Entities) confirms that the company identifier is correct, valid, and authorized to operate. If the verification is successful, the API sends the data for the creation of a wallet linked to the CNPJ. The API then stores this information and returns the operator's authorization token.

Below, we present an example of the JSON sent for the operator check-in.

Note that the *CNPJ* sent in the JSON will correspond to the *operator_id* identified in the table.

Listing 1: Example JSON Operator checkin request

```

{
  "vat_number": "123.345.678/999-11",
  "email": "company@companya.com.br",
  "name": "Company A",
  "version" : "basic2025a"
}
  
```

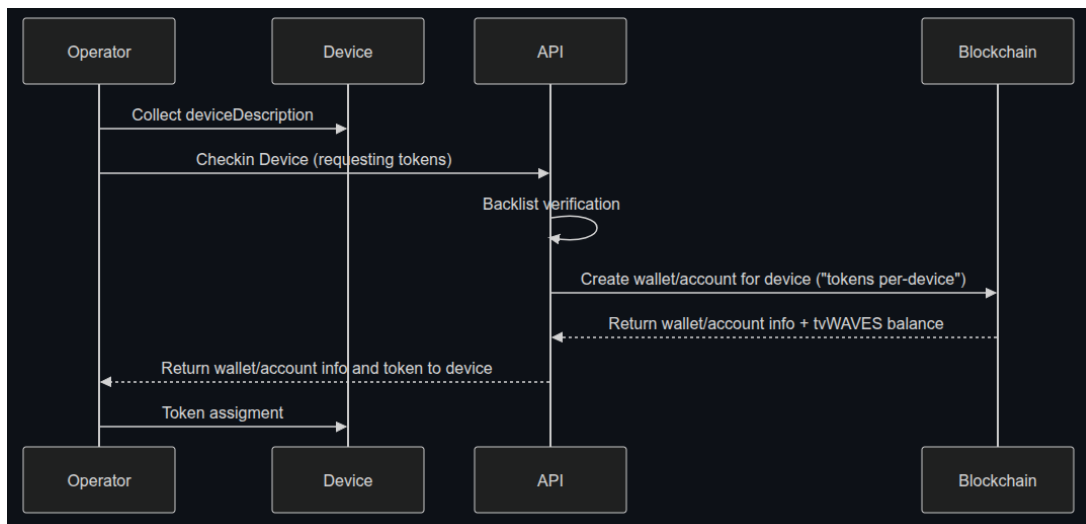
Response:

Listing 2: Example JSON Operator checkin response

```
{
  "success": true,
  "message": "Operator successfully authorized.",
  "token": "22bd803f-a953-489d-a311-e7e40f5d87a9",
  "id": 6
}
```

4.3.5. Device Token Assignment

The allocation of tokens to devices is requested by the operator and is currently performed on a per-device basis. The flow illustrated in the figure 8 shows the process of requesting and assigning tokens for each device.



Figures 8: Flow for requesting and assigning authentication tokens to consume the TVWS API

The operator collects information from the device, such as those found in *deviceDescription*¹³. This information is sent to the API, where a verification of the device data is performed. If the verification is successful, the API requests the creation of a wallet for the device on the blockchain. Finally, the account information and device tokens are sent to the operator, who then assigns the token to the device.

Below, we present an example of the JSON sent by the operator for the device checkin.

Listing 3: Example JSON Device checkin

```
{
  "operator_id": "123.345.678/999-11",
  "token": "22bd803f-a953-489d-a311-e7e40f5d87a9"
}
```

¹³<https://www.rfc-editor.org/rfc/rfc7545.html#page-37>

```

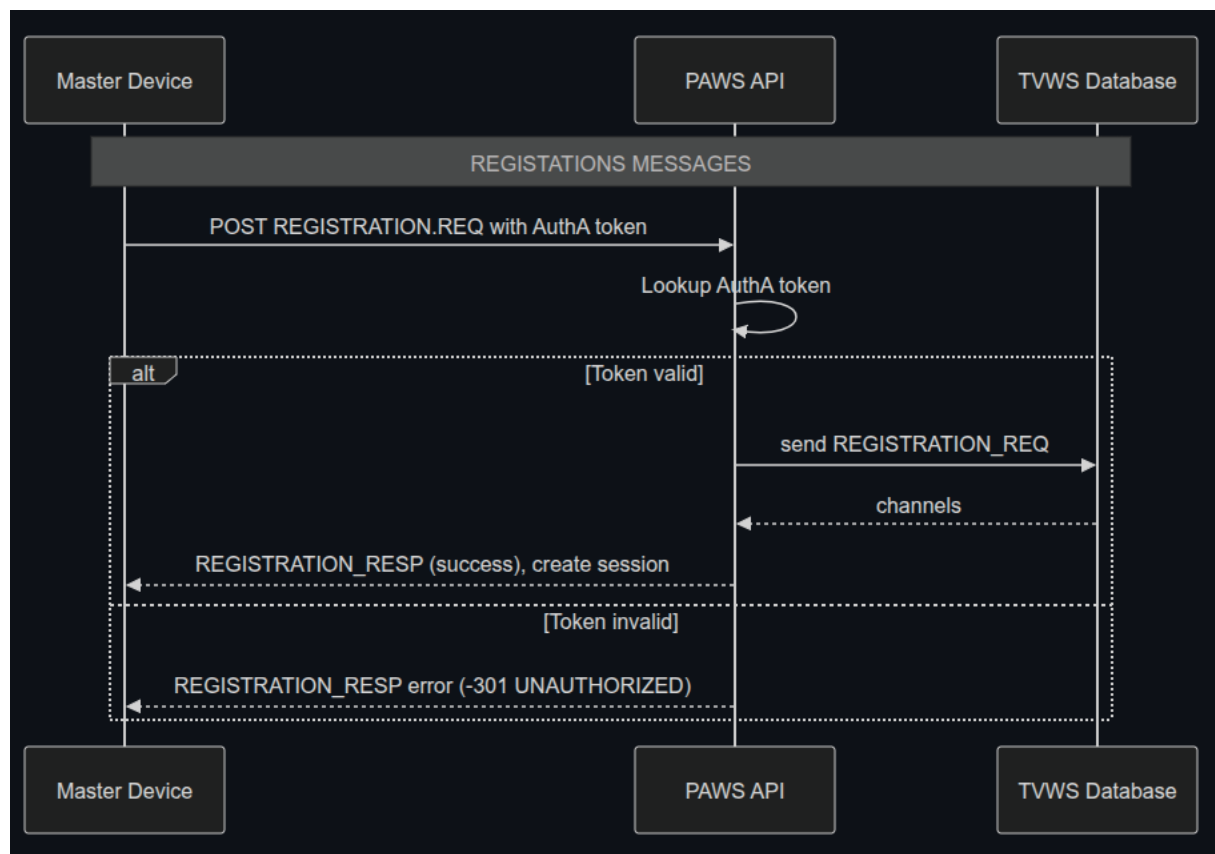
"deviceDesc":{
  "serialNumber":"4fbf-bae3-c8c5706745e3",
  "fccId":"94d8610a-9eea",
  "manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3",
  "modelId":"xpto-123-123-123",
  "rulesetIds":[
    "FccTvBandWhiteSpace-2010"
  ]
},
"version":"basic2025a"
}

```

4.3.6. Authentication and PAWS messages

In this session, we will present the authentication process during PAWS message exchanges. **REGISTRATION_REQ**

During registration, the master device sends its registration request using the AuthA token. The API validates the token, performs lookups, and creates a session for the device.



Figures 9: PAWS: Registration Request Flow

4.3.7. Practical Example

We will now address apractical examples of using the token for authentication.

Equivalent Curl Command

General example:

```
curl -X POST URL_HERE \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{ ... }'
```

Full example

```
curl -X POST URL_HERE \
-H "Content-Type: application/json" \
-H "Authorization: Bearer <token>" \
-d '{"jsonrpc":"2.0","method":"spectrum.paws.register","params":{"type":"REGISTRATION_REQ","version":"1.0","deviceDesc":{"serialNumber":"4fbf-bae3-c8c5706745e3","fccId":"94d8610a-9eea","manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3","modelId":"xpto-123-123-123","rulesetIds":["FccTvBandWhiteSpace-2010"]},"location":{"point":{"center":{"latitude":-4.978806,"longitude":-39.056607}}},"deviceOwner":{"owner":{"vcard":[{"version",{},"text","4.0"},["fn",{},"text","Fractal Networks"],["org",{},"text","Fractal Networks"],["email",{},"text","contact@fractalfn.com"],["tel",{},"uri","tel:+1-415-555-0100"]]},"operator":{"vcard":[{"version",{},"text","4.0"},["fn",{},"text","Jane Doe"],["org",{},"text","Fractal Networks"],["email",{},"text","jane.doe@fractalfn.com"],["tel",{},"uri","tel:+1-415-555-0101"]]}},"id":"10"}'
```

4.3.8. AVAIL_SPECTRUM_REQ

The following flow describes how master and slave devices interact when querying for available spectrum channels. Token validation ensures that all devices belong to the same operator wallet.

5 Administrative Services

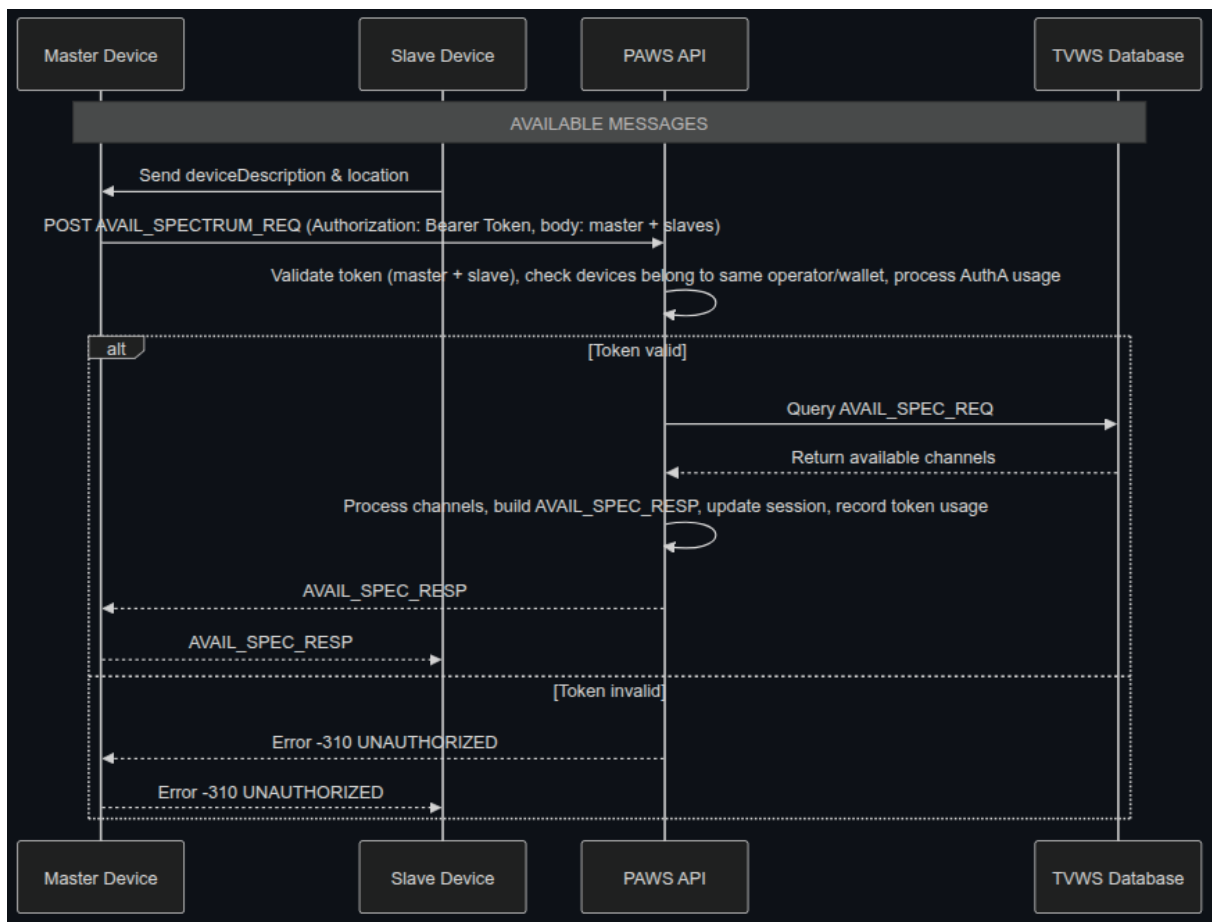
Some administrative services are provided through endpoints that allow other capabilities of the API to be utilized. The following are some examples.

GET /admin/test

Description: Health check to verify that the server is running.

Response:

```
1 "up"
```



Figures 10: PAWS: Registration Request Flow

GET /admin/get?location_id=pixel

Query Parameters:

- location_id (string)

Description: Retrieve blockchain location data and available channels.

Example: /admin/get?location_id=71fce8a17ac51196b06d57187f2c7ad9

Response:

```

1 {
2   "loc_id": "cf73bc...",
3   "location_data": {
4     "id": "statedId",
5     "cityId": "cityId",
6     "topLeft": [latitude, longitude],
7     "bottomRight": [latitude, longitude],
8     "sinr": [],
9     "tvNoiseFloor": [],

```

```

10   "protectedChannels": [],
11   "tvTransmitters": []
12 },
13   "available_channels": [... ]
14 }

```

GET /admin/get-location/:lat/:lon

Path Parameters:

- lat (float)
- lon (float)

Description: Find location ID by latitude and longitude.

Response:

```

1 {"pixel": "location_id" }

```

GET /admin/location/all

Description: Get a list of all registered location IDs.

Response:

```

1 {
2   "location_list": {
3     "id1": "up",
4     "id2": "up"
5   }
6 }

```

GET /admin/location/all/detail

Description: Get detailed information for all locations.

Response:

```

1 {
2   "location_list": {
3     "784962210b4f31c33154f38510400866": {
4       "cityId": 149,
5       "top_left": {
6         "latitude": -5.0295,
7         "longitude": -38.7975
8       },
9       "bottom_right": {
10        "latitude": -5.04,

```

```

11         "longitude": -38.808
12     },
13     "sinr": [0,0,0,-2.91, 3.61],
14     "protected_channels": [false, false]
15 }
16 }
17 }

```

GET /admin/lock

Query Parameters:

- location_id (string)
- channel (int)

Description: Lock a channel in a specific location on-chain.

Response:

```

1 {
2   "location_id": "cf73bc...",
3   "status": "200",
4   "transaction_id": "...",
5   "transaction_url": "https://explorer.solana.com/tx/..."
6 }

```

GET /admin/transmitter/all

Description: List all transmitters and their properties.

Response:

```

1 {
2   "mosaico_id": {
3     "pda": "...",
4     "status": "...",
5     "class": "...",
6     "channel": "...",
7     "position": "...",
8     "height": "...",
9     "hz": "...",
10    "erp": "..."
11  }
12 }

```

POST /admin/path

Description: Compute all pixels along a straight line between two points and return elevation.

Request Body:

```
1 {  
2   "point_a": {"longitude": -xx, "latitude": yy },  
3   "point_b": {"longitude": -xx, "latitude": yy }  
4 }
```

Response:

```
1 [  
2   {  
3     "pixel_id": "pixel_1",  
4     "latitude": xxx,  
5     "longitude": yyy,  
6     "height": 57.12  
7   },  
8   ...  
9 ]
```

POST /admin/hrpath

Description: Calculate terrain elevation between two points using higher resolution.

Request Body:

```
1 {  
2   "point_a": {"longitude": -xx, "latitude": yy },  
3   "point_b": {"longitude": -xx, "latitude": yy }  
4 }
```

Response:

```
1 [  
2   {  
3     "pixel_id": "-",  
4     "latitude": yy,  
5     "longitude": -xx,  
6     "height": 45.32  
7   },  
8   ...  
9 ]
```

GET /admin/session

Description: Retrieve all session records stored in the database.

Response:

```

1  [
2    {
3      "sid": "1741770350716",
4      "sess": {"route": "/paws" },
5      "created_at": "2025-03-12T09:05:50.714Z",
6      "expire": "2025-03-12T09:07:50.716Z",
7      "device": {
8        "serialNumber": "df7016ed-bc8d-4884-aace-e621de8a98e2",
9        "fccId": "ABCYXYN000C60",
10       "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",
11       "modelId": "xpto-123-123-123"
12     },
13     "spectrumspecs": [
14       {
15         "rulesetInfo": {
16           "authority": "ANATEL-BR",
17           "rulesetId": [
18             "FccTvBandWhiteSpace-2010"
19           ]
20         },
21         "spectrumSchedules": [
22           {
23             "spectra": [
24               {
25                 "profiles": [
26                   [{"hz": 54000000, "dbm": 14.375} ]
27                 ],
28                 "resolutionBwHz": 6000000
29               }
30             ],
31             "eventTime": {
32               "stopTime": "2025-03-12T09:07:50Z",
33               "startTime": "2025-03-12T09:05:50Z"
34             }
35           }
36         ],
37         "needsSpectrumReport": true
38       }
39     ],
40     "type": ["AVAIL_SPECTRUM_REQ"]
41   }
42 ]

```

GET /admin/list-tables**Description:** List all PostgreSQL tables in the public schema.**Response:**

```
1 [
2   {"table_name": "geography_columns" },
3   {"table_name": "geometry_columns" },
4   {"table_name": "spatial_ref_sys" },
5   {"table_name": "session" },
6   {"table_name": "telecom_location" },
7   {"table_name": "tvws_dbs" }
8 ]
```

6 Deploy

This section discusses two important aspects of the deployment and is not intended as a guide on how to perform the deployment; that information is provided in the code documentation.

6.1 Configuration file

The `default.json` file defines the global configuration parameters of the system. It centralizes operational, infrastructural, and functional settings required for the correct execution of the API and its associated services. Each top-level key is described below.

6.1.1. blockchain

This section contains all parameters related to blockchain interaction. It defines authentication credentials and external references required for on-chain operations. The configuration includes the network endpoint used to communicate with the blockchain and the private key employed for signing transactions, as well as the base URL of the blockchain transaction explorer used for monitoring and verification purposes.

```
1 "blockchain": {
2   "auth": {
3     "net": "http://ip:8899",
4     "privateKey": [...]
5   },
6   "urlSolanaExplorerTx": "https://explorer.solana.com/tx/"
7 }
```

The `blockchain` section defines all parameters required for interaction with the underlying blockchain infrastructure.

- `auth`: Groups authentication-related parameters.

- `net`: Specifies the blockchain network endpoint used to submit and query transactions.
- `privateKey`: Represents the private key used to cryptographically sign blockchain transactions. This key must be securely stored and restricted, as it grants full signing authority.
- `urlSolanaExplorerTx`: Defines the base URL of the blockchain explorer used to inspect and audit submitted transactions.

6.1.2. `server`

The `server` section defines API-level operational parameters and external service URLs. It specifies the port on which the API is exposed, monitoring and alerting mechanisms (including email-based notifications), and the base URLs used to access administrative services and propagation-related modules.

```
1 "server": {  
2   "apiParameterization": {  
3     "apiPort": 6000,  
4     "monitoringApi": {  
5       "email": {  
6         "serviceAlert": true,  
7         "emailUser": "...",  
8         "emailPassword": "...",  
9         "emailToAlert": "..."  
10      }  
11    }  
12  },  
13  "urlParametrization": {  
14    "urlBaseServer": "...",  
15    "urlPathAdminService": "...",  
16    "urlPropagationModuleMaxPower": "..."  
17  }  
18 }
```

The `server` section controls runtime behavior and external connectivity of the API.

- `apiParameterization`: Defines core API runtime settings.
 - `apiPort`: TCP port on which the API service is exposed.
 - `monitoringApi.email`: Configures email-based alerting.
 - `serviceAlert`: Enables or disables alert notifications.
 - `emailUser` and `emailPassword`: Credentials used by the alerting service.
 - `emailToAlert`: Recipient address for alerts.
- `urlParametrization`: Specifies external service endpoints required by the API, including administrative services and propagation-related modules.

6.1.3. authentication

This section controls the authentication and authorization behavior of the system. It defines whether authentication is enabled for operators, devices, and administrative users, as well as the expiration times associated with each authentication context. These parameters allow fine-grained control over access security and session validity.

```
1 "authentication": {  
2   "authentication_activation": false,  
3   "timeExpireAuthOperator": 240,  
4   "timeExpireAuthDevice": 300,  
5   "admin_authentication_activation": false,  
6   "ExpireAuthAdmin": 3600  
7 }
```

This section defines authentication policies and expiration rules.

- `authentication_activation`: Enables or disables authentication for operators and devices.
- `timeExpireAuthOperator`: Authentication validity period for operators, in seconds.
- `timeExpireAuthDevice`: Authentication validity period for devices, in seconds.
- `admin_authentication_activation`: Enables or disables administrative authentication.
- `ExpireAuthAdmin`: Expiration time for administrative authentication sessions.

6.1.4. database

The database configuration specifies the connection parameters for the geospatial database used by the system. It includes credentials, host address, port, and database name, enabling persistent storage and retrieval of geographic and operational data.

```
1 "database": {  
2   "geoDatabase": {  
3     "user": "postgres",  
4     "password": "postgres",  
5     "host": "localhost",  
6     "port": 5432,  
7     "database": "tvws"  
8   }  
9 }
```

The database section defines the connection parameters for the geospatial database.

- `geoDatabase`: Contains credentials and network parameters used to connect to the PostgreSQL database responsible for storing geographic and operational data.

6.1.5. test

This section contains parameters related to testing and performance evaluation. In particular, it allows the activation of execution time measurement for different API components, supporting diagnostics and performance analysis during development or validation phases.

```
1 "test": {  
2   "timeMetrics": {  
3     "description": "...",  
4     "recordExecTime": false  
5   }  
6 }
```

This section supports testing and diagnostics.

- **timeMetrics**: Controls execution time measurement.
- **recordExecTime**: When enabled, records performance metrics for internal API components.

6.1.6. propagationModule

The **propagationModule** section defines parameters associated with radio propagation modeling and power calculation. It includes numerical parameters used internally by the propagation algorithms, as well as the configuration of an external propagation service, such as its URL, signal-to-interference thresholds, and maximum transmission power constraints.

```
1 "propagationModule": {  
2   "propagationParameters": {  
3     "numSplits": 10,  
4     "channelSinMax": 27,  
5     "distancePoints": 30,  
6     "maxTxPower": 30  
7   },  
8   "propagationService": {  
9     "url": "...",  
10    "max_tx_power": 30,  
11    "sinr_border": 27,  
12    "query_distance_in_km": 30  
13  }  
14 }
```

The **propagationModule** section configures radio propagation analysis.

- **propagationParameters**: Defines internal algorithmic parameters such as SINR thresholds, distance resolution, and maximum transmission power.
- **propagationService**: Specifies the external propagation service endpoint and operational limits used during power and coverage calculations.

6.1.7. antennaParameters

This section defines configuration limits related to antenna modeling. In particular, it specifies the maximum supported size for antenna radiation pattern arrays, ensuring consistency and preventing oversized data structures during processing.

```
1 "antennaParameters": {  
2   "maxSizeArrays": 72  
3 }
```

This section defines constraints related to antenna modeling.

- `maxSizeArrays`: Maximum supported size for antenna radiation pattern arrays.

6.1.8. standardResponseMessage

The `standardResponseMessage` section defines default values used in PAWS-compliant responses generated by the API. It includes protocol versioning, ruleset information, spectrum constraints, bandwidth resolution, and API version identification. These parameters ensure standardized and interoperable responses across different PAWS interactions.

```
1 "standardResponseMessage": {  
2   "jsonrpc": "2.0",  
3   "version": "1.0",  
4   "rulesetInfos": {... },  
5   "rulesetInfo": {... },  
6   "needsSpectrumReport": true,  
7   "maxTotalBwHz": 6000000,  
8   "spectra": {"resolutionBwHz": 6000000},  
9   "versionApi": "2025.11a"  
10 }
```

This section defines default PAWS-compliant response parameters, including protocol versions, regulatory rulesets, bandwidth constraints, and API versioning. These values ensure interoperability and regulatory compliance across spectrum access interactions.

6.1.9. session

This section controls session management behavior. It defines cryptographic secrets, expiration times for different session types, cookie lifetime, session pruning intervals, and whether session management is enabled. These parameters govern how client sessions are created, maintained, and invalidated.

```
1 "session": {  
2   "session_secret": "...",  
3   "time_expire_session_avail": 120,  
4   "time_expire_session_init": 180,  
5   "time_expire_session_registration": 180,  
6   "time_expire_cookie": 60000,  
7   "pruneSessionInterval": 60,  
8 }
```

```
8 "session_activation": false
9 }
```

This section governs session lifecycle management.

- Cryptographic secrets and expiration times define how long sessions remain valid.
- `pruneSessionInterval` controls how frequently expired sessions are removed.
- `session_activation` enables or disables session tracking globally.

6.1.10. csv

The `csv` section defines formatting parameters used when processing CSV files. It specifies delimiters for fields, lists, and lines, ensuring consistent parsing and generation of CSV-based data exchanges.

```
1 "csv": {
2   "delimiter": ",",
3   "list_separator": ";",
4   "line_separator": "\n"
5 }
```

The `csv` section defines formatting rules for CSV file parsing and generation, ensuring consistent handling of structured text-based data.

6.2 Testing

The API ships with a suite of 72 automated tests written in **Jest**. They are organised across four files and can be executed with the following commands:

```
npm test # full suite
npm run test-paws # PAWS protocol tests
npm run test-admin # administrative route tests
npm run test-authorization # operator/device authentication tests
npm run test-db # database integrity tests
```

Before running the suite, copy and fill in the test configuration file:

```
cp test/config-test.example.json test/config-test.json
```

The subsections below describe each test case individually.

PAWS tests (test/paws.test.js) — 50 tests

Server health check.

1. **Server is up.** Sends GET `/paws/test` and verifies that the server responds with the string `üp` and HTTP 200.

Valid PAWS message flow.

2. **INIT_REQ accepted.** Posts a well-formed INIT_REQ and checks that the response contains a result field and that the response id matches the request id.
3. **REGISTRATION_REQ accepted.** Posts a well-formed REGISTRATION_REQ and verifies result and matching id.
4. **AVAIL_SPECTRUM_REQ accepted.** Posts a well-formed AVAIL_SPECTRUM_REQ and verifies that the response id matches the request id.

Method / params.type conformity.

5. **getSpectrum + INIT_REQ mismatch.** Sends `method=spectrum.paws.getSpectrum` with `params.type=INIT_REQ` and expects error -202 on `params.type`.
6. **init + AVAIL_SPECTRUM_REQ mismatch.** Sends `method=spectrum.paws.init` with `params.type=AVAIL_SPECTRUM_REQ` and expects error -202 on `params.type`.
7. **register + INIT_REQ mismatch.** Sends `method=spectrum.paws.register` with `params.type=INIT_REQ` and expects error -202 on `params.type`.
8. **Unknown method.** Sends an unrecognised method (`spectrum.paws.unknown`) and expects error -202 on `params.type`.

Geographic coordinate validation.

9. **Integer coordinates accepted.** Sends latitude -4 and longitude -39 (both integers) and verifies that no error -202 is returned.
10. **Latitude boundary values accepted.** Sends latitude -90 and latitude 90 separately and verifies that neither triggers error -202.
11. **Longitude boundary values accepted.** Sends longitude -180 and longitude 180 separately and verifies that neither triggers error -202.
12. **Latitude above 90 rejected.** Sends latitude 91 and expects error -202 on both `params.location.point.center.latitude` and `longitude`.
13. **Latitude below -90 rejected.** Sends latitude -91 and expects error -202 on both coordinate fields.
14. **Longitude above 180 rejected.** Sends longitude 181 and expects error -202 on both coordinate fields.
15. **Longitude below -180 rejected.** Sends longitude -181 and expects error -202 on both coordinate fields.
16. **String coordinates rejected.** Sends coordinates as strings (4", 39") and expects error -202.

Antenna direction and height validation.

17. **Negative azimuth rejected.** Sends `azimuth=-1` and expects error -202 on `params.antenna.antennaAzimuth`.
18. **Azimuth above 360 rejected.** Sends `azimuth=361` and expects error -202 on `azimuth`.
19. **Zenith above 90 rejected.** Sends `zenith=91` and expects error -202 on `params.antenna.antennaZenith`.
20. **Zenith below -90 rejected.** Sends `zenith=-91` and expects error -202 on `zenith`.
21. **No direction and no radiation pattern accepted.** Omits both `antennaDirection` and `antennaRadiationPattern` and verifies that the request is accepted (both fields are optional when used together).
22. **Radiation pattern without direction rejected.** Omits `antennaDirection` while supplying `antennaRadiationPattern` and expects error -202 on `params.antenna.antennaDirection` (the pattern makes the direction mandatory).
23. **Missing height rejected.** Omits the `height` field and expects error -202 on `params.antenna.height`.
24. **Missing heightType rejected.** Omits the `heightType` field and expects error -202 on `params.antenna.heightType`.
25. **AGL height below -200 m rejected.** Sends `height=-201` with `heightType=AGL` and expects error -202 on `params.antenna.height`.
26. **AMSL height below minimum rejected.** Sends a height one unit below `minHeightAMSL` (default -3500 m) with `heightType=AMSL` and expects error -202 on `params.antenna.height`.
27. **AMSL height above maximum rejected.** Sends a height one unit above `maxHeightAMSL` (default 8848 m) with `heightType=AMSL` and expects error -202 on `params.antenna.height`.

Session expiry (conditional).

28. **Expired session creates a new session.** Only runs when `session.status = true` in `config-test.json`. Establishes a session via `AVAIL_SPECTRUM_REQ`, waits for `time_expire_session` + 10 s, then sends a second request and verifies that no error -302 or -106 is returned — confirming that an expired session is transparently replaced.

Radiation pattern validation (`antennaRadiationPattern`). *Null and empty patterns:*

29. **Null pattern accepted.** Sends `antennaRadiationPattern=null` and verifies the request is accepted (treated as isotropic).
30. **Empty array accepted.** Sends `antennaRadiationPattern=[]` and verifies the request is accepted (treated as isotropic).

Wrong type:

31. **Non-array pattern rejected.** Sends a string (`"invalid"`) instead of an array and expects error -202 on `params.antenna.antennaRadiationPattern`.

32. **Sub-array not an array rejected.** Sends a string in place of a sub-array and expects error -202.

Polarisation / sub-array count mismatch:

33. **Single polarisation with two sub-arrays rejected.** Sends `antennaPolarization=["v"]` with two sub-arrays and expects error -201.
34. **Dual polarisation with one sub-array rejected.** Sends `antennaPolarization=["v","h"]` with one sub-array and expects error -201.

Sub-array size:

35. **Fewer than 72 elements accepted.** Sends a sub-array shorter than 72 elements and verifies the request is accepted (the server replaces it with a zero array).
36. **More than 72 elements accepted.** Sends a sub-array with 73 elements and verifies the request is accepted (replaced with zero).

Element type validation:

37. **Non-numeric element rejected.** Places a string ("bad") at position 10 of a 72-element array and expects error -202.
38. **NaN element rejected.** Places NaN at position 0 of a 72-element array and expects error -202.

Valid patterns:

39. **Single polarisation with 72 valid elements accepted.** Sends one sub-array of exactly 72 values, all ≤ 0 dBi, and verifies the request is accepted.
40. **Dual polarisation with two valid sub-arrays accepted.** Sends two sub-arrays of exactly 72 values each (all ≤ 0 dBi) and verifies the request is accepted.

Value constraint (≤ 0 dBi):

41. **All values at exactly 0 dBi accepted (boundary).** Sends `Array(72).fill(0)` and verifies the request is accepted.
42. **All negative values accepted.** Sends `Array(72).fill(-3.0)` and verifies the request is accepted.
43. **Value of 0.001 dBi at intermediate position rejected.** Sets position 5 to 0.001 and expects error -202.
44. **Positive value at first position rejected.** Sets position 0 to 1.5 and expects error -202.
45. **Positive value in second sub-array rejected (dual polarisation).** First sub-array is fully valid; second sub-array has position 35 set to 0.5. Expects error -202.

Antenna gain validation (antennaGain).

46. **Gain above 50 dBi rejected.** Sends `antennaGain=50.001` and expects error `-202` on `params.antenna.antennaGain`.
47. **Negative gain rejected.** Sends `antennaGain=-0.001` and expects error `-202`.
48. **Non-numeric gain rejected.** Sends `antennaGain="high"` and expects error `-202`.
49. **Boundary values 0 and 50 dBi accepted.** Sends both boundary values and verifies that neither returns error `-202`.
50. **Absent antennaGain accepted.** Sends a request without the field and verifies the request is accepted (field is optional).

Antenna polarisation validation (antennaPolarization).

51. **No polarisation and no radiation pattern accepted.** Omits both `antennaPolarization` and `antennaRadiationPattern` and verifies the request is accepted (both fields are optional when absent together).

Administrative tests (test/admin.test.js) — 9 tests

52. **Admin server is up.** Sends `GET /admin/test` and verifies HTTP 200.
53. **Active sessions list.** Sends `GET /admin/session` and verifies that the response is an array. Only runs when `session.status = true` in `config-test.json`.
54. **Locations with full detail.** Sends `GET /admin/location/all/detail` and verifies HTTP 200 and the presence of the `location_list` field.
55. **Locations summary.** Sends `GET /admin/location/all` and verifies HTTP 200 and the presence of `location_list`.
56. **Pixel lookup by coordinates.** Sends `GET /admin/get-location/{lat}/{lon}` using the coordinates from `config-test.json` and verifies that the response contains a `pixel` field.
57. **Transmitter list.** Sends `GET /admin/transmitter/all` and verifies HTTP 200 and an object response.
58. **HR Path calculation.** Posts `POST /admin/hrpath` with two geographic points (A and B from `config-test.json`) and verifies HTTP 200.
59. **Path calculation.** Posts `POST /admin/path` with the same two points and verifies HTTP 200.
60. **Database table list.** Sends `GET /admin/list-tables` and verifies that the response array contains all expected tables: `session`, `authorization`, `telecom_location`, `tvws_dbs`, `geography_columns`, `geometry_columns`, and `spatial_ref_sys`.

Authentication tests (test/auth.test.js) — 4 tests

61. **Set operator token expiry.** Posts POST /admin/setTime/authOperator/240 and verifies success: true.
62. **Set device token expiry.** Posts POST /admin/setTime/authDevice/300 and verifies success: true.
63. **Operator check-in.** Posts POST /admin/checkin-operator with VAT number, name, e-mail, and version; verifies success: true, token, and id in the response; stores the token for the next test.
64. **Device check-in.** Uses the token from the previous test to post POST /admin/checkin-device with serial number, model ID, and manufacturer ID; verifies success: true.

Database integrity tests (test/db.test.js) — 8 tests

These tests connect directly to PostgreSQL (bypassing the HTTP layer) and require running `npm run migrate` before the first execution.

Table session:

65. **Table exists.** Queries `information_schema.tables` and asserts that the session table is present in the public schema.
66. **Insert record.** Inserts a row with `sid`, `sess`, `device`, and `location` and verifies `rowCount=1` and the returned `sid`.
67. **Find inserted record.** Queries by `sid` and verifies exactly one row is returned.
68. **Delete record.** Deletes by `sid` and confirms the row no longer exists (`rowCount=0`).

Table authorization:

69. **Table exists.** Queries `information_schema.tables` and asserts that the authorization table is present.
70. **Insert record.** Inserts a row with `token` (UUID), `scope`, `operator_id`, `contact`, and `version`; verifies `rowCount=1` and that `id` is defined.
71. **Find inserted record.** Queries by `token` and verifies exactly one row is returned.
72. **Delete record.** Deletes by `token` and confirms the row no longer exists (`rowCount=0`).

Referencias

Chen, V., Das, S., Zhu, L., Malyar, J., & McCann, P. (2015, mayo). *Protocol to Access White-Space (PAWS) Databases* (RFC N.º 7545). Internet Engineering Task Force. <https://doi.org/10.17487/RFC7545>